

ANSA Flasher MVP: Dependency Roadmap & Team Allocation

This document establishes a deterministic, gate-driven execution plan for the **ANSA Flasher MVP** based strictly on the requirements of the **ANSA Flasher Study PDF**. It organizes the 37 atomic tasks into sequential dependency chains across **five staging gates** and distributes them strategically between **two team members**.

Part 1: Strategic Team Roles

To minimize communication overhead and prevent integration bottlenecks, the project is divided into two distinct engineering roles:

Developer A: Systems & Silicon Lead (Rust & Firmware)

- **Focus:** Low-level OS interactions, Rust background threads, CLI execution wrappers, STM32 C/C++ firmware development, and serial data pipelines.
- **Why this role exists:** Hardware flashing is highly coupled with firmware behavior. By assigning both the STM32-side peripheral configuration and the Rust-side flasher logic to a single developer, we eliminate cross-team friction on physical communication boundaries.
- **Scope:** 20 Tasks (12 Backend, 8 Firmware)

Developer B: Product & Release Lead (Frontend & DevOps/Infra)

- **Focus:** Tauri Webview UI development, Web Portal and Manifest hosting, static data schemas, code-signing certificates, and release packaging pipelines.
- **Why this role exists:** Building modern React/TypeScript user flows and managing cloud/automated distribution chains require a shared engineering mindset. Developer B can design and refine user interfaces using mock inputs while Developer A develops low-level APIs, and then step in to automate the final release packaging.
- **Scope:** 17 Tasks (11 Frontend, 6 Infrastructure)

Part 2: Gate-Driven Dependency Roadmap

The project is structured into **five sequential Gates**. Work within each gate can be done in parallel, but all tasks in a gate must be verified before proceeding to the next gate.

```

graph TD
  %% Gate 1: Registry
  subgraph Gate 1: Registry Gate [Foundation]
    T1_1[T1.1: Define Manifest Schema - Dev B] --> T1_2[T1.2: Populate Seed Manifest - Dev B]
    T1_3[T1.3: Set Up Web Hosting - Dev B]
    T1_4[T1.4: Design Download Webpage - Dev B] --> T1_5[T1.5: Build Release Routing - Dev B]
    T5_1[T5.1: Embed Crypto Public Key - Dev A]
  end

  %% Gate 2: Backbone
  subgraph Gate 2: Backbone Gate [Hardware & UI Discovery]
    T2_1[T2.1: Write USB Polling Module - Dev A] --> T2_2[T2.2: Implement ST-Link SWD ID - Dev A]
    T2_1 --> T2_3[T2.3: Implement USB DFU ID - Dev A]
    T2_2 & T2_3 --> T3_1[T3.1: Implement Tauri USB Cmds - Dev A]
    T3_1 --> T4_5[T4.5: Build UI Discovery Bar - Dev B]

    T6_1[T6.1: Initialize STM32 UART - Dev A] --> T6_2[T6.2: Print Boot Banner - Dev A]
    T6_5[T6.5: Configure LED GPIO Pins - Dev A] --> T6_6[T6.6: Implement LED Blink Loop - Dev A]
    T2_4[T2.4: Document UART Fallback - Dev B]
  end

  %% Gate 3: Data Integration
  subgraph Gate 3: Data Integration Gate [Verification & Serialization]
    T1_2 & T1_3 --> T3_2[T3.2: Implement Manifest Downloader - Dev A]
    T3_2 --> T3_3[T3.3: SHA-256 Checksum Validator - Dev A]
    T3_2 & T5_1 --> T3_4[T3.4: Crypto Signature Verifier - Dev A]

    T3_2 --> T4_1[T4.1: Build Board Picker UI - Dev B]
    T4_1 --> T4_2[T4.2: Build Version Dropdown - Dev B]
    T4_2 --> T4_6[T4.6: Implement Confirm Modal - Dev B]

    T4_3[T4.3: Design Param Form - Dev B] --> T4_4[T4.4: Serialize Config Blob - Shared]

    T3_5[T3.5: Bundle Static Flashing Binaries - Dev B/A]
    T3_5 --> T3_6[T3.6: Develop DFU-Util Wrapper - Dev A]
    T3_5 --> T3_7[T3.7: Develop ST-Link Wrapper - Dev A]
    T3_6 & T3_7 --> T4_7[T4.7: Write Progress Parser - Dev A]
    T4_7 --> T4_8[T4.8: Connect Progress Event Bridge - Dev A]
    T4_8 --> T4_9[T4.9: Build Progress Screen UI - Dev B]
  end

  %% Gate 4: Physical Validation
  subgraph Gate 4: Physical Validation Gate [Flashing & Verification]
    T4_9 --> T4_10[T4.10: Build Success Screen - Dev B]
    T4_10 --> T4_12[T4.12: Build UI UART Console - Dev B]
    T4_11[T4.11: Write Host UART Listener - Dev A] --> T4_12

    T6_2 --> T6_3[T6.3: Implement UART CLI Parser - Dev A] --> T6_4[T6.4: Implement about Command - Dev A]
    T6_6 --> T6_7[T6.7: Program LED Boot Success - Dev A]
    T6_6 --> T6_8[T6.8: Program LED System Fault - Dev A]
  end

  %% Gate 5: Release
  subgraph Gate 5: Release Gate [OS Security & Distribution]
    T5_2[T5.2: Configure Win Authenticode - Dev B]
    T5_3[T5.3: Configure macOS Notarization - Dev B]
    T1_3 --> T5_4[T5.4: Set Up Auto-Updater Registry - Dev B] --> T5_5[T5.5: Configure Auto-Updater Client - Dev B]
  end

  %% Gate Connections
  Gate 1: Registry Gate --> Gate 2: Backbone Gate
  Gate 2: Backbone Gate --> Gate 3: Data Integration Gate
  Gate 3: Data Integration Gate --> Gate 4: Physical Validation Gate
  Gate 4: Physical Validation Gate --> Gate 5: Release Gate

```

Gate 1: The Registry Gate (Foundation)

Objective: Build the schemas, landing portals, and public hosting nodes. This sets up the remote source of truth for the flasher's configuration.

Task ID	Task Title	Primary Owner	Target Deliverable	Parallel / Sequential Dependencies
T1.1	Define JSON Manifest Schema	Developer B	manifest.schema.json	None (Independent start)
T1.2	Populate Seed Manifest File	Developer B	Seed manifest.json	Requires T1.1
T1.3	Set Up Web Hosting & Storage	Developer B	Active HTTPS web bucket URLs	None (Independent start)
T1.4	Design OS-Detecting Landing Page	Developer B	Web portal HTML/JS code	None (Independent start)
T1.5	Build Release Routing	Developer B	Functional landing page router	Requires T1.4
T5.1	Embed Crypto Public Key	Developer A	Hardcoded public key in Rust module	None (Independent start)

Gate 2: The Backbone Gate (Hardware & UI Discovery)

Objective: Implement background hardware monitoring and STM32 boot initialization. By the end of this gate, the UI will change dynamically when a hardware board is plugged in.

Task ID	Task Title	Primary Owner	Target Deliverable	Parallel / Sequential Dependencies
T2.1	Write USB Polling Module (Rust)	Developer A	Background thread usb_monitor.rs	Requires Gate 1 completed
T2.2	Implement ST-Link SWD ID	Developer A	SWD VID/PID filtering in Rust	Requires T2.1
T2.3	Implement USB DFU ID	Developer A	ROM DFU VID/PID filtering in Rust	Requires T2.1
T2.4	Document UART Fallback	Developer B	Physical hookup guide (UART_FALLBACK.md)	None (Can run in parallel)
T3.1	Implement Tauri USB Cmds	Developer A	commands.rs mapping to Frontend	Requires T2.2 and T2.3
T4.5	Build UI Discovery Bar	Developer B	DeviceStatusBar.tsx React component	Requires T3.1 (Frontend listens to Rust events)
T6.1	Initialize STM32 UART	Developer A	USART config in firmware main.c	Requires Gate 1 completed
T6.2	Print Boot Banner	Developer A	Transmit routine on STM32 startup	Requires T6.1
T6.5	Configure LED GPIO Pins	Developer A	Pin mapping configurations in gpio.c	Requires Gate 1 completed
T6.6	Implement LED Blink Loop	Developer A	Timer-driven loop in status_led.c	Requires T6.5

Gate 3: The Data Integration Gate (Verification & Serialization)

Objective: Download firmware, verify it cryptographically, serialize custom configuration pages, and configure CLI tool wrappers to pipe live progress.

Task ID	Task Title	Primary Owner	Target Deliverable	Parallel / Sequential Dependencies
T3.2	Implement Manifest Downloader	Developer A	Rust downloader module downloader.rs	Requires T1.2 and T1.3
T3.3	SHA-256 Checksum Validator	Developer A	Hash checking module verifier.rs	Requires T3.2
T3.4	Crypto Signature Verification	Developer A	Signature checking in verifier.rs	Requires T3.2 and T5.1
T3.5	Bundle Static Flashing Binaries	Developer B	Platform-specific executables under binaries/	Requires Gate 2 completed
T3.6	Develop DFU-Util Shell Wrapper	Developer A	Rust execution wrapper dfu_wrapper.rs	Requires T3.5
T3.7	Develop ST-Link Shell Wrapper	Developer A	Rust execution wrapper swd_wrapper.rs	Requires T3.5
T4.1	Build Board Selection Screen	Developer B	Card picker grid UI (BoardPicker.tsx)	Requires T3.2 (Uses manifest data)
T4.2	Build Version Dropdown Picker	Developer B	Version picker UI (VersionPicker.tsx)	Requires T4.1
T4.3	Design Customization Form	Developer B	Input form UI (CustomizationForm.tsx)	None (Can run in parallel)
T4.4	Serialize Config Blob	Shared	Custom binary layout block serializer	Requires T4.3
T4.6	Implement Confirm Modal	Developer B	Warn-and-confirm dialog ConfirmationModal.tsx	Requires T4.2
T4.7	Write Progress Output Parser	Developer A	Text regex matching inside dfu_wrapper.rs	Requires T3.6 and T3.7
T4.8	Connect Progress Event Bridge	Developer A	Webview event emission in main.rs	Requires T4.7
T4.9	Build Progress Screen UI	Developer B	Live progress bar interface (FlashingProgress.tsx)	Requires T4.8

Gate 4: The Physical Validation Gate (Flashing & Post-Flash Verification)

Objective: End-to-end flashing onto real target microcontrollers. The UI drives the flash, receives success events, listens to the target UART boot-up banner, and visualizes success via LEDs.

Task ID	Task Title	Primary Owner	Target Deliverable	Parallel / Sequential Dependencies
T4.10	Build Success Screen UI	Developer B	SuccessScreen.tsx component	Requires T4.9 (Transition upon flash success)
T4.11	Write Host UART Listener	Developer A	Rust serial port listener uart_listener.rs	Requires Gate 3 completed
T4.12	Build UI UART Console	Developer B	Serial log viewer and checklist component	Requires T4.10 and T4.11
T6.3	Implement UART CLI Parser	Developer A	Non-blocking command processor cli.c	Requires T6.1
T6.4	Implement UART "about" Cmd	Developer A	CLI action mapped to print version detail	Requires T6.3
T6.7	Program LED Boot Success	Developer A	Success state call in initialization hook	Requires T6.6 and T6.2
T6.8	Program LED System Fault	Developer A	Fault state call inside standard error handlers	Requires T6.6

Gate 5: The Release Gate (OS Security & Distribution)

Objective: Establish secure distribution. Wrap the polished tool in platform installers, configure automated code-signing and notarization keys, and set up the updater system.

Task ID	Task Title	Primary Owner	Target Deliverable	Parallel / Sequential Dependencies
T5.2	Configure Win Authenticode	Developer B	Signed installer files (.msi) in CI pipeline	Requires Gate 4 verified
T5.3	Configure macOS Notarization	Developer B	Signed/notarized DMG installer in CI pipeline	Requires Gate 4 verified
T5.4	Set Up Auto-Updater Registry	Developer B	Update endpoint schema and signature registry	Requires T1.3
T5.5	Configure Auto-Updater Client	Developer B	Active update polling loop inside Tauri	Requires T5.4

Part 4: Load Balancing & Integration Checkpoints

[Developer A Total Workload] ██████████ (20 Tasks)
[Developer B Total Workload] ██████████ (17 Tasks)

To maintain execution speed, the team should synchronize at **three critical integration handshakes**:

1. **The Discovery Handshake (End of Gate 2):** Developer A hands over the Tauri events that signal hardware status (ST-Link connected , DFU device connected). Developer B binds these signals directly to the interactive DeviceStatusBar in the frontend.

2. **The Serialization Handshake (Mid-Gate 3):** Developer B designs the parameter UI form. Together, Developer A and B define the exact binary layout (e.g., specific byte alignment for Drone ID and WiFi strings) so the serializing code (written by B or shared) outputs the exact format Developer A's backend will flash to the reserved page.
3. **The Progress Handshake (End of Gate 3):** Developer A finishes parsing the stdout streams of `dfu-util` and `STM32_Programmer_CLI` and fires numeric percentage events. Developer B hooks these up to the animated `FlashingProgress` UI component.

...

Developer A (Systems & Silicon Lead) → Uddin

Uddin is the perfect fit for Developer A.

- **Why his background fits:** He has direct, hands-on experience interfacing sensors with microcontrollers at the **register level**, has built a minor project using **STM32 with HAL**, and worked on an industrial IoT project using Arduino.
- **His assignment:** Uddin will own the **STM32 C/C++ firmware side** (USART, Status LED loops, and the command line parser) and the **Rust system-level backend** (managing USB device polling via `libusb` / `rusb`, handling the `dfu-util` and SWD wrapper utilities, and building the host-side UART listener).
- **Why this makes sense:** Uddin's experience with physical MCU registers and HAL means he understands clock setups, pinouts, and interrupt routines. This directly translates to the firmware tasks in Gate 2 and Gate 4.

Developer B (Product & Release Lead) → Praprara

You are the perfect fit for Developer B.

- **Why your background fits:** You have a proven track record of using AI to rapidly build **web applications**, **mobile apps** (Android), and **IoT projects with data streaming to web interfaces**.
- **Your assignment:** You will own the **Tauri frontend (React/TypeScript UI)**, designing the board and version pickers, building the parameter serialization forms, and managing the **infrastructure and DevOps pipeline** (manifest hosting, web landing page, Windows Authenticode, macOS Notarization, and the Tauri auto-update registry).
- **Why this makes sense:** Building Tauri Webviews is fundamentally web and application development. Since you've already successfully built web and mobile apps with AI, handling React states, JSON schemas, web-to-backend communication, and build/release pipelines is right in your wheelhouse.

The Collaboration Dynamic (How you work together)

Because both of you are leveraging AI, this division of labor will keep you from stepping on each other's toes:

1. **Independent Workflows:** Uddin can work purely in C (using STM32CubeIDE) and pure Rust backend modules, while you work in React/TS and Tauri configuration. You won't have to merge conflicting code in the same files.
2. **Clean Handshakes:**
 - **In Gate 2:** Uddin writes the Rust USB detection. He tells you what event names to listen for (`dfu-connected`), and you make your UI status bar react to them.
 - **In Gate 3:** You design the custom Wifi/Drone ID parameter form. Together, you will lock down a simple data layout (e.g., "The first 4 bytes are the Drone ID, followed by 32 bytes for the SSID"). You write the JS-to-Rust serializer, and Uddin's backend writes those exact bytes to the reserved flash page.

Here is your immediate game plan for **Gate 1** and **Gate 2**:

Your Immediate Actions (Praprara - Developer B)

You can start laying down the product foundation immediately:

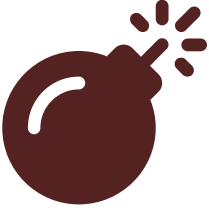
1. **Draft the Manifest Schema (T1.1 & T1.2):** Create your `manifest.json` using the template we pulled from the study. This acts as your software's remote database.
2. **Spin Up the Download Landing Page (T1.4 & T1.5):** Build the static React or vanilla HTML landing page with OS-detection. Since you've built web apps and streamed data before, this will be highly straightforward.
3. **Deploy Static Web Hosting (T1.3):** Set up a simple storage bucket (like AWS S3 or Cloudflare R2) to host your manifest and placeholder binaries.

Uddin's Immediate Actions (Uddin - Developer A)

Uddin can begin building the physical hardware backbone:

1. **Initialize STM32 UART & Status LED (T6.1, T6.2, T6.5 & T6.6):** Spin up STM32CubeMX, configure a USART TX/RX peripheral, map status LED GPIOs, and write the initial non-blocking start loop that transmits the ASCII boot banner.
2. **Write the Rust USB Polling Module (T2.1, T2.2 & T2.3):** Initialize the Tauri-Rust backend and use the `rusb` crate to monitor USB connections. You have the exact hardware identifiers needed for filtering:
 - **ST-Link SWD:** Vendor ID `0483` , Product IDs `3748` / `374B` / `374E`
 - **USB DFU Mode:** Vendor ID `0483` , Product ID `DF11`

You have a pristine set of instructions, a clear division of labor, and a solid technical roadmap. Go build an amazing utility!



Syntax error in text
mermaid version 11.16.1



Syntax error in text
mermaid version 11.16.1



Syntax error in text
mermaid version 11.16.1